Name-Patil Amruta

Roll no-2317088

Batch-'A'

sub-mushrooms

In [348…
```
import pandas as pd
```

In [350…
```
import seaborn as sns
```

In [352…
```
import numpy as np
```

In [354…
```
import matplotlib.pyplot as plt
```

In [356…
```
#Attribute Information: (classes: edible=e, poisonous=p)
#cap-shape: bell=b,conical=c,convex=x,flat=f, knobbed=k,sunken=s
#cap-surface: fibrous=f,grooves=g,scaly=y,smooth=s
#cap-color: brown=n,buff=b,cinnamon=c,gray=g,green=r,pink=p,purple=u,red=e,white=w,ye
#bruises: bruises=t,no=f
#odor: almond=a,anise=l,creosote=c,fishy=y,foul=f,musty=m,none=n,pungent=p,spicy=s
#gill-attachment: attached=a,descending=d,free=f,notched=n
#gill-spacing: close=c,crowded=w,distant=d
#gill-size: broad=b,narrow=n
#gill-color: black=k,brown=n,buff=b,chocolate=h,gray=g, green=r,orange=o,pink=p,purpl
#stalk-shape: enlarging=e,tapering=t
#stalk-root: bulbous=b,club=c,cup=u,equal=e,rhizomorphs=z,rooted=r,missing=?
#stalk-surface-above-ring: fibrous=f,scaly=y,silky=k,smooth=s
#stalk-surface-below-ring: fibrous=f,scaly=y,silky=k,smooth=s
#stalk-color-above-ring: brown=n,buff=b,cinnamon=c,gray=g,orange=o,pink=p,red=e,white
#stalk-color-below-ring: brown=n,buff=b,cinnamon=c,gray=g,orange=o,pink=p,red=e,white
#veil-type: partial=p,universal=u
#veil-color: brown=n,orange=o,white=w,yellow=y
#ring-number: none=n,one=o,two=t
#ring-type: cobwebby=c,evanescent=e,flaring=f,large=l,none=n,pendant=p,sheathing=s,zo
#spore-print-color: black=k,brown=n,buff=b,chocolate=h,green=r,orange=o,purple=u,whit
#population: abundant=a,clustered=c,numerous=n,scattered=s,several=v,solitary=y
#habitat: grasses=g,leaves=l,meadows=m,paths=p,urban=u,waste=w,woods=d
```

In [358…
```
df=pd.read_csv("mushrooms.csv")
```

In [360…
```
df
```

Out[360…

	class	cap-shape	cap-surface	cap-color	bruises	odor	gill-attachment	gill-spacing	gill-size	gill-color	…
0	p	x	s	n	t	p	f	c	n	k	…
1	e	x	s	y	t	a	f	c	b	k	…
2	e	b	s	w	t	l	f	c	b	n	…
3	p	x	y	w	t	p	f	c	n	n	…
4	e	x	s	g	f	n	f	w	b	k	…
…	…	…	…	…	…	…	…	…	…	…	…
8119	e	k	s	n	f	n	a	c	b	y	…
8120	e	x	s	n	f	n	a	c	b	y	…
8121	e	f	s	n	f	n	a	c	b	n	…
8122	p	k	y	n	f	y	f	c	n	b	…
8123	e	x	s	n	f	n	a	c	b	y	…

8124 rows × 23 columns

In [362…

```
df.head()
```

Out[362…

	class	cap-shape	cap-surface	cap-color	bruises	odor	gill-attachment	gill-spacing	gill-size	gill-color	…	su
0	p	x	s	n	t	p	f	c	n	k	…	
1	e	x	s	y	t	a	f	c	b	k	…	
2	e	b	s	w	t	l	f	c	b	n	…	
3	p	x	y	w	t	p	f	c	n	n	…	
4	e	x	s	g	f	n	f	w	b	k	…	

5 rows × 23 columns

In [364…

```
df.tail()
```

Out[364…

	class	cap-shape	cap-surface	cap-color	bruises	odor	gill-attachment	gill-spacing	gill-size	gill-color	…
8119	e	k	s	n	f	n	a	c	b	y	…
8120	e	x	s	n	f	n	a	c	b	y	…
8121	e	f	s	n	f	n	a	c	b	n	…
8122	p	k	y	n	f	y	f	c	n	b	…
8123	e	x	s	n	f	n	a	c	b	y	…

5 rows × 23 columns

In [366…

```
from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
```

In [368…

```
for col in df.select_dtypes(include='object').columns:
```

```
    df[col] = le.fit_transform(df[col])
```

In [370…

```
df
```

Out[370…

	class	cap-shape	cap-surface	cap-color	bruises	odor	gill-attachment	gill-spacing	gill-size	gill-color	…
0	1	5	2	4	1	6	1	0	1	4	…
1	0	5	2	9	1	0	1	0	0	4	…
2	0	0	2	8	1	3	1	0	0	5	…
3	1	5	3	8	1	6	1	0	1	5	…
4	0	5	2	3	0	5	1	1	0	4	…
…	…	…	…	…	…	…	…	…	…	…	…
8119	0	3	2	4	0	5	0	0	0	11	…
8120	0	5	2	4	0	5	0	0	0	11	…
8121	0	2	2	4	0	5	0	0	0	5	…
8122	1	3	3	4	0	8	1	0	1	0	…
8123	0	5	2	4	0	5	0	0	0	11	…

8124 rows × 23 columns

In [372…

```
df.isnull()
```

Out[372…

	class	cap-shape	cap-surface	cap-color	bruises	odor	gill-attachment	gill-spacing	gill-size	gill-color	…
0	False	False	False	False	False	False	False	False	False	False	..
1	False	False	False	False	False	False	False	False	False	False	..
2	False	False	False	False	False	False	False	False	False	False	..
3	False	False	False	False	False	False	False	False	False	False	..
4	False	False	False	False	False	False	False	False	False	False	..
…	…	…	…	…	…	…	…	…	…	…	..
8119	False	False	False	False	False	False	False	False	False	False	..
8120	False	False	False	False	False	False	False	False	False	False	..
8121	False	False	False	False	False	False	False	False	False	False	..
8122	False	False	False	False	False	False	False	False	False	False	..
8123	False	False	False	False	False	False	False	False	False	False	..

8124 rows × 23 columns

In [ ]:

In [375…

```
df.isnull().sum()
```

Out[375…

```
class                       0
cap-shape                   0
cap-surface                 0
cap-color                   0
bruises                     0
odor                        0
gill-attachment             0
gill-spacing                0
gill-size                   0
gill-color                  0
stalk-shape                 0
stalk-root                  0
stalk-surface-above-ring    0
stalk-surface-below-ring    0
stalk-color-above-ring      0
stalk-color-below-ring      0
veil-type                   0
veil-color                  0
ring-number                 0
ring-type                   0
spore-print-color           0
population                  0
habitat                     0
dtype: int64
```

In [377…

```
df.isnull().count()
```

Out[377…

```
class                       8124
cap-shape                   8124
cap-surface                 8124
cap-color                   8124
bruises                     8124
odor                        8124
gill-attachment             8124
gill-spacing                8124
gill-size                   8124
gill-color                  8124
stalk-shape                 8124
stalk-root                  8124
stalk-surface-above-ring    8124
stalk-surface-below-ring    8124
stalk-color-above-ring      8124
stalk-color-below-ring      8124
veil-type                   8124
veil-color                  8124
ring-number                 8124
ring-type                   8124
spore-print-color           8124
population                  8124
habitat                     8124
dtype: int64
```

In [379…

```
df.dropna(axis=0,inplace=True)
```

In [381…

```
df
```

Out[381…

	class	cap-shape	cap-surface	cap-color	bruises	odor	gill-attachment	gill-spacing	gill-size	gill-color	…
0	1	5	2	4	1	6	1	0	1	4	…
1	0	5	2	9	1	0	1	0	0	4	…
2	0	0	2	8	1	3	1	0	0	5	…
3	1	5	3	8	1	6	1	0	1	5	…
4	0	5	2	3	0	5	1	1	0	4	…
…	…	…	…	…	…	…	…	…	…	…	…
8119	0	3	2	4	0	5	0	0	0	11	…
8120	0	5	2	4	0	5	0	0	0	11	…
8121	0	2	2	4	0	5	0	0	0	5	…
8122	1	3	3	4	0	8	1	0	1	0	…
8123	0	5	2	4	0	5	0	0	0	11	…

8124 rows × 23 columns

In [383…

```
df.isnull().sum()
```

Out[383…

```
class                     0
cap-shape                 0
cap-surface               0
cap-color                 0
bruises                   0
odor                      0
gill-attachment           0
gill-spacing              0
gill-size                 0
gill-color                0
stalk-shape               0
stalk-root                0
stalk-surface-above-ring  0
stalk-surface-below-ring  0
stalk-color-above-ring    0
stalk-color-below-ring    0
veil-type                 0
veil-color                0
ring-number               0
ring-type                 0
spore-print-color         0
population                0
habitat                   0
dtype: int64
```

In [385…

```
df.dropna()
```

Out[385…

	class	cap-shape	cap-surface	cap-color	bruises	odor	gill-attachment	gill-spacing	gill-size	gill-color	…
0	1	5	2	4	1	6	1	0	1	4	…
1	0	5	2	9	1	0	1	0	0	4	…
2	0	0	2	8	1	3	1	0	0	5	…
3	1	5	3	8	1	6	1	0	1	5	…
4	0	5	2	3	0	5	1	1	0	4	…
…	…	…	…	…	…	…	…	…	…	…	…
8119	0	3	2	4	0	5	0	0	0	11	…
8120	0	5	2	4	0	5	0	0	0	11	…
8121	0	2	2	4	0	5	0	0	0	5	…
8122	1	3	3	4	0	8	1	0	1	0	…
8123	0	5	2	4	0	5	0	0	0	11	…

8124 rows × 23 columns

In [387…

```
df.shape
```

Out[387…

```
(8124, 23)
```

df.size

In [389…

```
df.ndim
```

Out[389…

```
2
```

In [399…

```
df.describe()
```

Out[399…

	class	cap-shape	cap-surface	cap-color	bruises	odor	a
count	8124.000000	8124.000000	8124.000000	8124.000000	8124.000000	8124.000000	8
mean	0.482029	3.348104	1.827671	4.504677	0.415559	4.144756	
std	0.499708	1.604329	1.229873	2.545821	0.492848	2.103729	
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
25%	0.000000	2.000000	0.000000	3.000000	0.000000	2.000000	
50%	0.000000	3.000000	2.000000	4.000000	0.000000	5.000000	
75%	1.000000	5.000000	3.000000	8.000000	1.000000	5.000000	
max	1.000000	5.000000	3.000000	9.000000	1.000000	8.000000	

8 rows × 23 columns

In [401…

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8124 entries, 0 to 8123
Data columns (total 23 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   class                     8124 non-null   int64
 1   cap-shape                 8124 non-null   int64
 2   cap-surface               8124 non-null   int64
 3   cap-color                 8124 non-null   int64
 4   bruises                   8124 non-null   int64
 5   odor                      8124 non-null   int64
 6   gill-attachment           8124 non-null   int64
 7   gill-spacing              8124 non-null   int64
 8   gill-size                 8124 non-null   int64
 9   gill-color                8124 non-null   int64
 10  stalk-shape               8124 non-null   int64
 11  stalk-root                8124 non-null   int64
 12  stalk-surface-above-ring  8124 non-null   int64
 13  stalk-surface-below-ring  8124 non-null   int64
 14  stalk-color-above-ring    8124 non-null   int64
 15  stalk-color-below-ring    8124 non-null   int64
 16  veil-type                 8124 non-null   int64
 17  veil-color                8124 non-null   int64
 18  ring-number               8124 non-null   int64
 19  ring-type                 8124 non-null   int64
 20  spore-print-color         8124 non-null   int64
 21  population                8124 non-null   int64
 22  habitat                   8124 non-null   int64
dtypes: int64(23)
memory usage: 1.4 MB
```

In [403…

```
B=df.select_dtypes('category')
B
```

Out[403…

0
1
2
3
4
...
8119
8120
8121
8122
8123

8124 rows × 0 columns

In [405…

```
df ['class'].unique()
```

Out[405…

```
array([1, 0])
```

In [407…

```
df ['cap-shape'].nunique()
```

Out[407…

```
6
```

In [409…

```
df['cap-shape'].count()
```

Out[409… 8124

In [411…
```
df['cap-shape'].value_counts()
```

Out[411…
```
cap-shape
5    3656
2    3152
3     828
0     452
4      32
1       4
Name: count, dtype: int64
```

In [413…
```
def detect_outliers_iqr(data):
    outliers = {}
    for col in data.select_dtypes(include=['number']).columns:
        Q1 = np.percentile(data[col], 25, interpolation='midpoint')
        Q3 = np.percentile(data[col], 75, interpolation='midpoint')
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        outliers[col] = data[(data[col] < lower_bound) | (data[col] > upper_bound)][c
    return outliers
```

In [425…
```
outliers = detect_outliers_iqr(df)
print(outliers)
```

```
{'class': Series([], Name: class, dtype: int64), 'cap-shape': Series([], Name: cap-sha
pe, dtype: int64), 'cap-surface': Series([], Name: cap-surface, dtype: int64), 'cap-co
lor': Series([], Name: cap-color, dtype: int64), 'bruises': Series([], Name: bruises,
dtype: int64), 'odor': Series([], Name: odor, dtype: int64), 'gill-attachment': 6038
0
6040    0
6375    0
6424    0
6434    0
       ..
8115    0
8119    0
8120    0
8121    0
8123    0
Name: gill-attachment, Length: 210, dtype: int64, 'gill-spacing': 4       1
14      1
16      1
29      1
35      1
       ..
8094    1
8096    1
8099    1
8109    1
8111    1
Name: gill-spacing, Length: 1312, dtype: int64, 'gill-size': Series([], Name: gill-siz
e, dtype: int64), 'gill-color': Series([], Name: gill-color, dtype: int64), 'stalk-sha
pe': Series([], Name: stalk-shape, dtype: int64), 'stalk-root': 0       3
3       3
4       3
8       3
13      3
       ..
2817    3
2844    3
2881    3
2890    3
2966    3
Name: stalk-root, Length: 1312, dtype: int64, 'stalk-surface-above-ring': Series([], N
ame: stalk-surface-above-ring, dtype: int64), 'stalk-surface-below-ring': Series([], N
ame: stalk-surface-below-ring, dtype: int64), 'stalk-color-above-ring': 305     3
384     3
449     3
873     3
890     3
       ..
7984    4
7986    4
8001    4
8095    1
8114    1
Name: stalk-color-above-ring, Length: 1588, dtype: int64, 'stalk-color-below-ring': 63
2     3
944     3
963     3
1030    3
1043    3
       ..
7984    4
7986    4
8001    4
8095    1
8114    1
Name: stalk-color-below-ring, Length: 1652, dtype: int64, 'veil-type': Series([], Nam
e: veil-type, dtype: int64), 'veil-color': 6038    0
```

```
6040    0
6375    1
6424    0
6434    0
       ..
8115    1
8119    1
8120    0
8121    1
8123    1
Name: veil-color, Length: 200, dtype: int64, 'ring-number': 3984    2
4106    2
4283    2
4291    2
4357    2
       ..
8096    2
8099    2
8109    2
8111    2
8114    0
Name: ring-number, Length: 636, dtype: int64, 'ring-type': Series([], Name: ring-type,
dtype: int64), 'spore-print-color': Series([], Name: spore-print-color, dtype: int64),
'population': 4       0
14      0
16      0
56      0
65      0
       ..
8107    1
8114    1
8119    1
8121    1
8123    1
Name: population, Length: 724, dtype: int64, 'habitat': 3984    6
4283    6
4291    6
4357    6
4380    6
       ..
5939    6
5953    6
5961    6
5976    6
5996    6
Name: habitat, Length: 192, dtype: int64}
```

In [ ]:

LOGISTIC REGRESSION

In [429…
```
import sklearn as sk
```

In [431…
```
x=df.drop(columns='class')
y=df['class']
```

In [433…
```
from sklearn.model_selection import train_test_split
```

In [435…
```
x_train,x_test,y_train,y_test=train_test_split(x,y,train_size=0.8,test_size=0.2,rando
```

In [437…
```
len(x_train)
```

Out[437… 6499

In [439…
```
len(y_train)
```

Out[439… 6499

In [441…
```
len(x_test)
```

Out[441… 1625

In [443…
```
len(y_test)
```

Out[443… 1625

In [445…
```
x_train.shape
```

Out[445… (6499, 22)

In [447…
```
y_train.shape
```

Out[447… (6499,)

In [449…
```
x_test.shape
```

Out[449… (1625, 22)

In [451…
```
y_test.shape
```

Out[451… (1625,)

In [453…
```
from sklearn.linear_model import LogisticRegression
model=LogisticRegression()
```

In [455…
```
model
```

Out[455…
```
▾ LogisticRegression
LogisticRegression()
```

In [457…
```
model.fit(x_train,y_train)
```

```
/home/admin1/anaconda3/lib/python3.9/site-packages/sklearn/linear_model/_logistic.py:4
60: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

Out[457…
```
▾ LogisticRegression
LogisticRegression()
```

In [459…
```
y_pred=model.predict(x_test)
```

In [461…
```
y_pred
```

Out[461… array([0, 1, 0, ..., 1, 0, 0])

In [463…
```
model.score(x_test,y_test)
```

Out[463…
```
0.9526153846153846
```

In [465…
```
from sklearn.metrics import accuracy_score
```

In [467…
```
logistic_accu=accuracy_score(y_test,y_pred)
```

In [469…
```
logistic_accu
```

Out[469…
```
0.9526153846153846
```

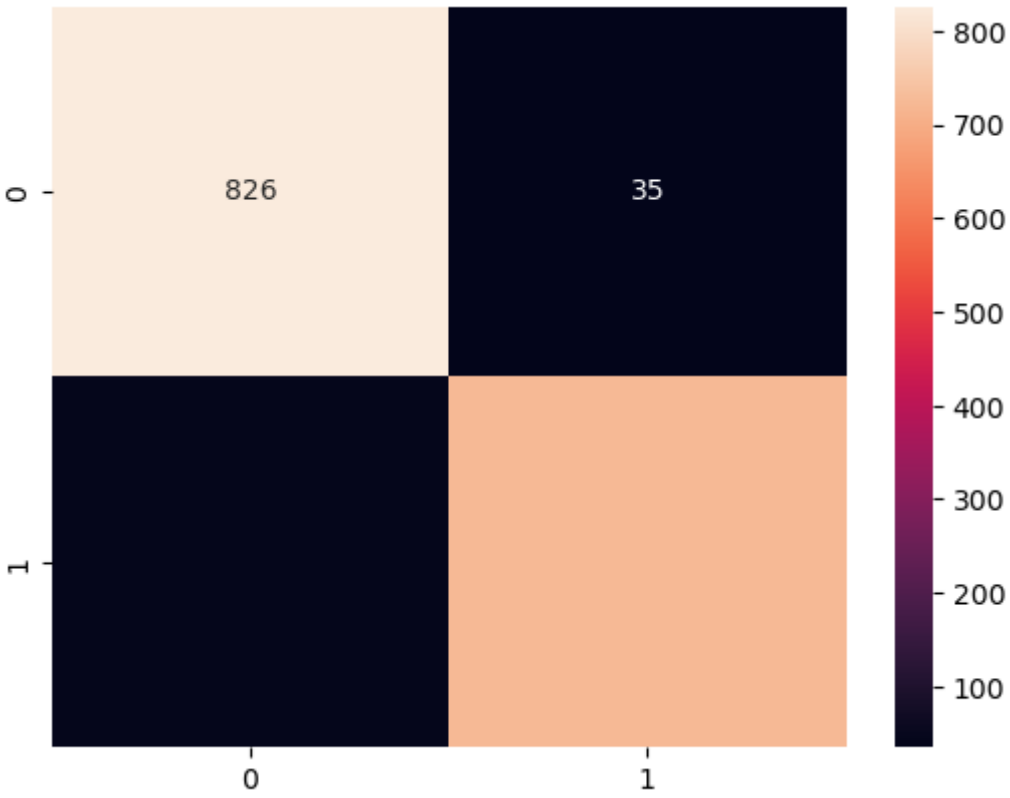
In [471…
```
from sklearn.metrics import confusion_matrix,accuracy_score
from sklearn.preprocessing import StandardScaler
```

In [473…
```
cm=confusion_matrix(y_test,y_pred)
cm
```

Out[473…
```
array([[826,  35],
       [ 42, 722]])
```

In [475…
```
sns.heatmap(cm,annot=True,fmt='g')
```

Out[475…
```
<Axes: >
```



In [477…
```
cm.ravel()
```

Out[477…
```
array([826,  35,  42, 722])
```

In [479…
```
tn , fp ,fn , tp= confusion_matrix(y_test,y_pred).ravel()
print(f'True Positive (TP): {tp}')
print(f'True Negative (TN): {tn}')
print(f'False Positive (FP): {fp}')
print(f'False Negative (FN): {fn}')
```

```
True Positive (TP): 722
True Negative (TN): 826
False Positive (FP): 35
False Negative (FN): 42
```

In [481…
```
accuracy=(tp+tn)/(tp+tn+fp+fn)
```

In [483…
```
accuracy
```

Out[483… 0.9526153846153846

In [485…
```
from sklearn.metrics import classification_report
print(classification_report(y_test,y_pred))
```

```
              precision    recall  f1-score   support

           0       0.95      0.96      0.96       861
           1       0.95      0.95      0.95       764

    accuracy                           0.95      1625
   macro avg       0.95      0.95      0.95      1625
weighted avg       0.95      0.95      0.95      1625
```

In [487…
```
from sklearn.metrics import precision_score
```

In [489…
```
print(precision_score(y_test,y_pred))
```

```
0.9537648612945839
```

In [491…
```
recall=tp/(tp+fn)
```

In [493…
```
print(recall)
```

```
0.9450261780104712
```

In [495…
```
precision=tp/(tp+fp)
```

In [497…
```
precision
```

Out[497… 0.9537648612945839

In [499…
```
f1_score=2*((precision*recall)/(precision+recall))
```

In [501…
```
print(f1_score)
```

```
0.9493754109138725
```

NAIVE BAYES

In [504…
```
import sklearn as sk
```

In [506…
```
x=df.drop(columns='class')
y=df['class']
```

In [508…
```
from sklearn.model_selection import train_test_split
```

In [510…
```
x_train,x_test,y_train,y_test=train_test_split(x,y,train_size=0.8,test_size=0.2,rando
```

In [512…
```
x_train
```

Out[512…

	cap-shape	cap-surface	cap-color	bruises	odor	gill-attachment	gill-spacing	gill-size	gill-color	stalk-shape	..
2321	5	3	2	1	5	1	0	0	10	1	..
6705	5	2	4	0	8	1	0	1	0	1	..
5583	5	3	4	0	8	1	0	1	0	1	..
5495	2	3	9	0	2	1	0	0	2	0	..
6819	2	2	4	0	8	1	0	1	0	1	..
...	...	...	...	...	...	...	...	...	...	...	..
3441	5	0	5	0	1	1	0	1	5	0	..
1344	2	2	8	0	5	1	1	0	5	1	..
4623	2	3	3	0	2	1	0	0	3	0	..
7293	3	2	3	0	5	1	1	0	10	0	..
1289	2	2	8	0	5	1	1	0	5	1	..

6499 rows × 22 columns

In [514…

```
y_train
```

Out[514…

```
2321    0
6705    1
5583    1
5495    1
6819    1
       ..
3441    1
1344    0
4623    1
7293    0
1289    0
Name: class, Length: 6499, dtype: int64
```

In [516…

```
x_test
```

Out[516…

	cap-shape	cap-surface	cap-color	bruises	odor	gill-attachment	gill-spacing	gill-size	gill-color	stalk-shape	..
582	5	3	8	1	3	1	0	0	4	0	..
2288	2	2	4	1	6	1	0	1	7	0	..
1789	5	2	8	0	5	1	1	0	5	1	..
3374	2	3	2	1	5	1	0	0	9	1	..
814	5	2	4	1	6	1	0	1	4	0	..
...	...	...	...	...	...	...	...	...	...	...	..
2572	5	3	4	1	5	1	0	0	10	1	..
5045	2	0	3	0	2	1	0	0	3	0	..
4995	2	3	9	0	2	1	0	0	3	0	..
3359	5	0	3	0	1	1	1	1	9	0	..
2729	5	3	4	1	5	1	0	0	9	1	..

1625 rows × 22 columns

In [518…
```
y_test
```

Out[518…
```
582     0
2288    1
1789    0
3374    0
814     1
       ..
2572    0
5045    1
4995    1
3359    1
2729    0
Name: class, Length: 1625, dtype: int64
```

In [520…
```
from sklearn.naive_bayes import GaussianNB
```

In [522…
```
model=GaussianNB()
```

In [524…
```
model
```

Out[524…
```
▾ GaussianNB
GaussianNB()
```

In [526…
```
model.fit(x_train,y_train)
```

Out[526…
```
▾ GaussianNB
GaussianNB()
```

In [528…
```
y_pred=model.predict(x_test)
```

In [530…
```
y_pred
```

Out[530…
```
array([0, 1, 0, ..., 1, 0, 0])
```

In [532…
```
model.score(x_test,y_test)
```

Out[532…
```
0.9224615384615384
```

In [534…
```
from sklearn.metrics import accuracy_score
```

In [536…
```
GaussiNB_accu=accuracy_score(y_test,y_pred)
```

In [538…
```
GaussiNB_accu
```

Out[538…
```
0.9224615384615384
```

In [540…
```
Error_rate=1-accuracy_score(y_test,y_pred)
```

In [542…
```
Error_rate
```

Out[542…
```
0.07753846153846156
```

In [544…
```
from sklearn.metrics import classification_report
```

In [546…
```
print(classification_report(y_test,y_pred))
```

```
              precision    recall  f1-score   support

           0       0.93      0.93      0.93       861
           1       0.92      0.92      0.92       764

    accuracy                           0.92      1625
   macro avg       0.92      0.92      0.92      1625
weighted avg       0.92      0.92      0.92      1625
```

In [548…
```
from sklearn.metrics import confusion_matrix
```

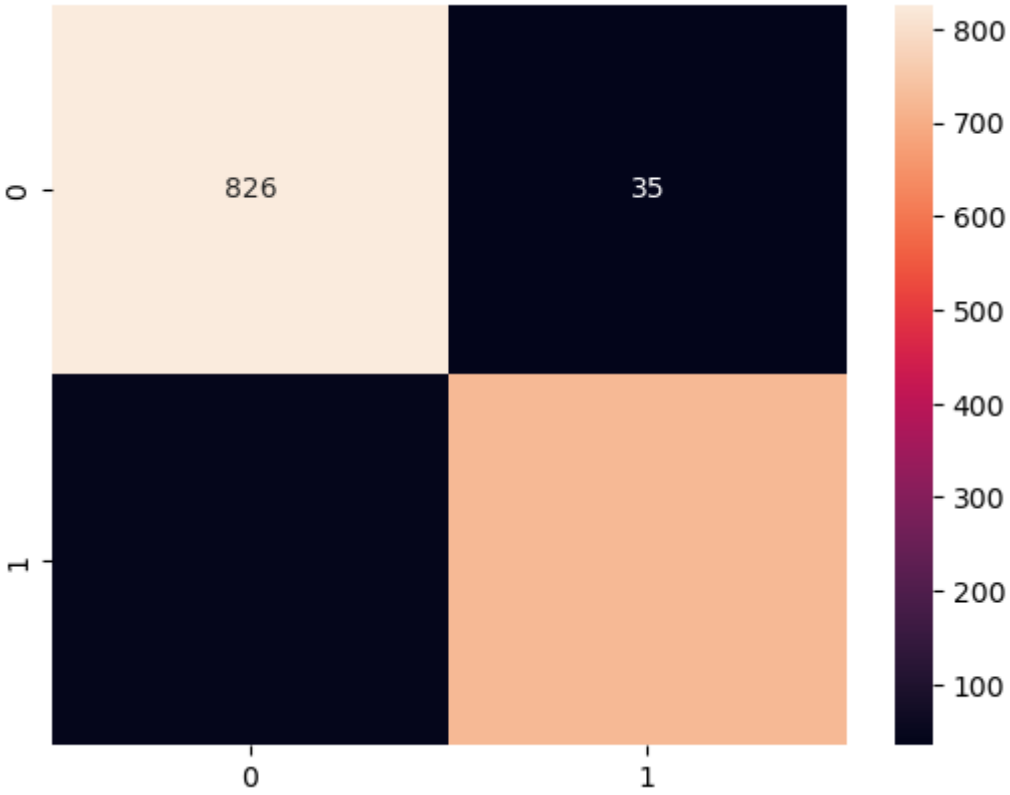
In [550…
```
CM=confusion_matrix(y_test,y_pred)
```

In [552…
```
CM
```

Out[552…
```
array([[797,  64],
       [ 62, 702]])
```

In [554…
```
sns.heatmap(cm,annot=True,fmt='g')
```

Out[554…
```
<Axes: >
```



In [556…
```
from sklearn.naive_bayes import MultinomialNB
```

In [558…
```
model=MultinomialNB()
```

In [560…
```
model
```

Out[560…
```
▾ MultinomialNB
MultinomialNB()
```

In [562…
```
model.fit(x_train,y_train)
```

Out[562…
```
▾ MultinomialNB
MultinomialNB()
```

In [564…
```
y_pred=model.predict(x_test)
```

In [566…
```
y_pred
```

Out[566…
```
array([0, 0, 0, ..., 1, 0, 0])
```

In [568…
```
model.score(x_test,y_test)
```

Out[568…
```
0.7895384615384615
```

In [570…
```
from sklearn.metrics import accuracy_score
```

In [572…
```
MultinomialNB_accu=accuracy_score(y_test,y_pred)
```

In [574…
```
MultinomialNB_accu
```

Out[574…
```
0.7895384615384615
```

In [576…
```
Error_rate=1-accuracy_score(y_test,y_pred)
```

In [578…
```
Error_rate
```

Out[578…
```
0.21046153846153848
```

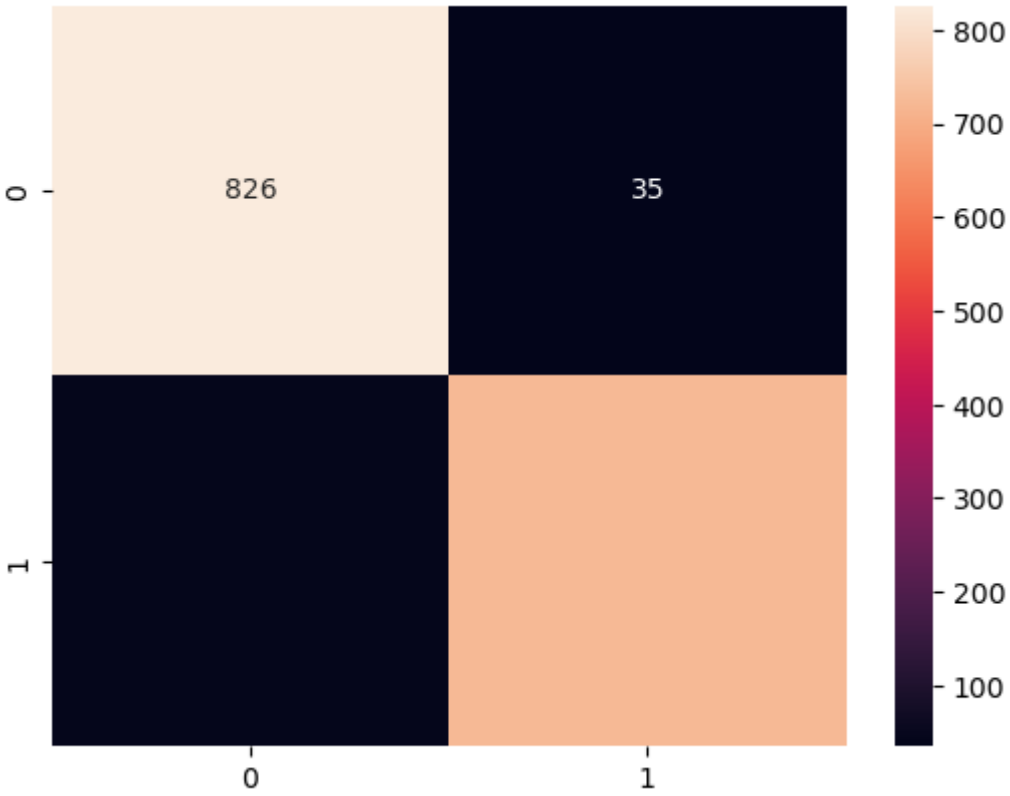
In [580…
```
from sklearn.metrics import confusion_matrix
```

In [582…
```
CM=confusion_matrix(y_test,y_pred)
CM
```

Out[582…
```
array([[809,  52],
       [290, 474]])
```

In [584…
```
sns.heatmap(cm,annot=True,fmt='g')
```

Out[584…
```
<Axes: >
```



In [586…
```
from sklearn.metrics import classification_report
```

In [588…
```
print(classification_report(y_test,y_pred))
```

```
              precision    recall  f1-score   support

           0       0.74      0.94      0.83       861
           1       0.90      0.62      0.73       764

    accuracy                           0.79      1625
   macro avg       0.82      0.78      0.78      1625
weighted avg       0.81      0.79      0.78      1625
```

In [590…
```
from sklearn.naive_bayes import BernoulliNB
```

In [592…
```
model=BernoulliNB()
```

In [594…
```
model
```

Out[594…
```
▾ BernoulliNB
BernoulliNB()
```

In [596…
```
model.fit(x_train,y_train)
```

Out[596…
```
▾ BernoulliNB
BernoulliNB()
```

In [598…
```
y_pred=model.predict(x_test)
```

In [600…
```
y_pred
```

Out[600…
```
array([0, 1, 0, ..., 0, 0, 0])
```

In [602…
```
model.score(x_test,y_test)
```

Out[602…
```
0.8295384615384616
```

In [604…
```
from sklearn.metrics import accuracy_score
```

In [606…
```
BernoulliNB_accu=accuracy_score(y_test,y_pred)
```

In [608…
```
BernoulliNB_accu
```

Out[608…
```
0.8295384615384616
```

In [610…
```
Error_rate=1-accuracy_score(y_test,y_pred)
```

In [612…
```
Error_rate
```

Out[612…
```
0.17046153846153844
```

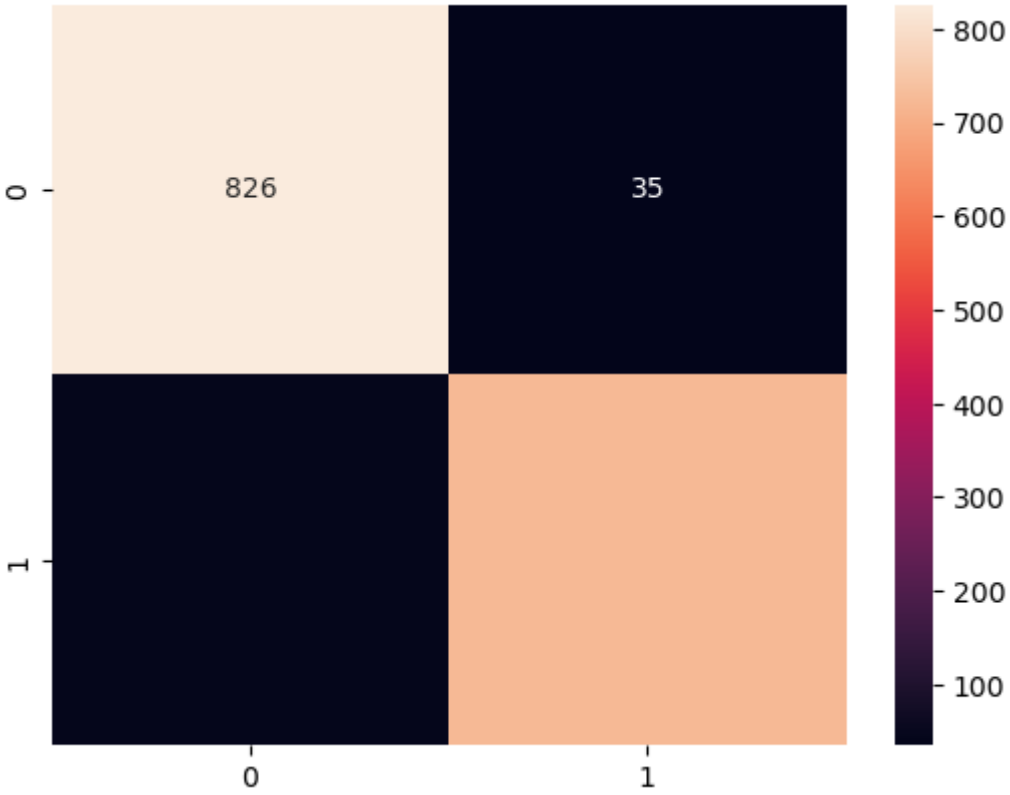
In [614…
```
from sklearn.metrics import confusion_matrix
```

In [616…
```
CM=confusion_matrix(y_test,y_pred)
CM
```

Out[616…
```
array([[796,  65],
       [212, 552]])
```

In [618…
```
sns.heatmap(cm,annot=True,fmt='g')
```

Out[618…
```
<Axes: >
```



In [620…
```
from sklearn.metrics import classification_report
```

In [622…
```
print(classification_report(y_test,y_pred))
```

```
              precision    recall  f1-score   support

           0       0.79      0.92      0.85       861
           1       0.89      0.72      0.80       764

    accuracy                           0.83      1625
   macro avg       0.84      0.82      0.83      1625
weighted avg       0.84      0.83      0.83      1625
```

Random forest

In [639…
```
from sklearn.ensemble import RandomForestClassifier
```

In [640…
```
rf_model1=RandomForestClassifier(random_state=42)
```

In [641…
```
rf_model1.fit(x_train,y_train)
```

Out[641…
```
▾ RandomForestClassifier
RandomForestClassifier(random_state=42)
```

In [642…
```
forest_pred=rf_model1.predict(x_test)
```

In [643…
```
forest_pred
```

Out[643…
```
array([0, 1, 0, ..., 1, 1, 0])
```

In [644…
```
rf_model1.score(x_test,y_pred)
```

Out[644…
```
0.8295384615384616
```

In [645…
```
rf_accuracy=accuracy_score(y_test,forest_pred)
```

In [646…
```
rf_accuracy
```

Out[646… 1.0

In [647…
```
forest_report=classification_report(y_test,forest_pred)
```

In [648…
```
print(forest_report)
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       861
           1       1.00      1.00      1.00       764

    accuracy                           1.00      1625
   macro avg       1.00      1.00      1.00      1625
weighted avg       1.00      1.00      1.00      1625
```

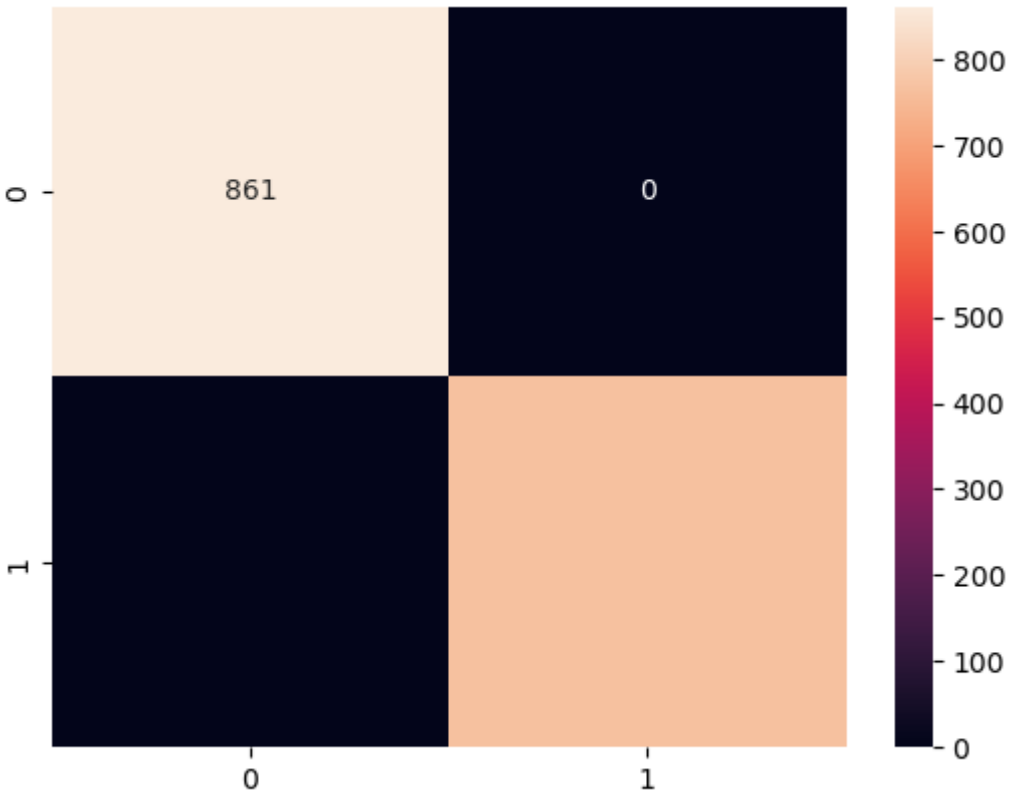
In [649…
```
forestcm=confusion_matrix(y_test,forest_pred)
```

In [650…
```
forestcm
```

Out[650…
```
array([[861,   0],
       [  0, 764]])
```

In [651…
```
sns.heatmap(forestcm,annot=True,fmt='g')
```

Out[651… <Axes: >



decision Tree

In [653…
```
from sklearn.tree import DecisionTreeClassifier
```

In [654…
```
tree=DecisionTreeClassifier()
```

In [655…
```
tree.fit(x_train,y_train)
```

Out[655…
```
▾ DecisionTreeClassifier
DecisionTreeClassifier()
```

In [657…
```
tree_pred=tree.predict(x_test)
```

In [658…
```
tree_pred
```

Out[658…
```
array([0, 1, 0, ..., 1, 1, 0])
```

In [659…
```
tree.score(x_test,y_test)
```

Out[659…
```
1.0
```

In [660…
```
Decision_accuracy=accuracy_score(y_test,tree_pred)
```

In [661…
```
Decision_accuracy
```

Out[661…
```
1.0
```

In [662…
```
report_tree=classification_report(y_test,tree_pred)
print(report_tree)
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00       861
           1       1.00      1.00      1.00       764

    accuracy                           1.00      1625
   macro avg       1.00      1.00      1.00      1625
weighted avg       1.00      1.00      1.00      1625
```

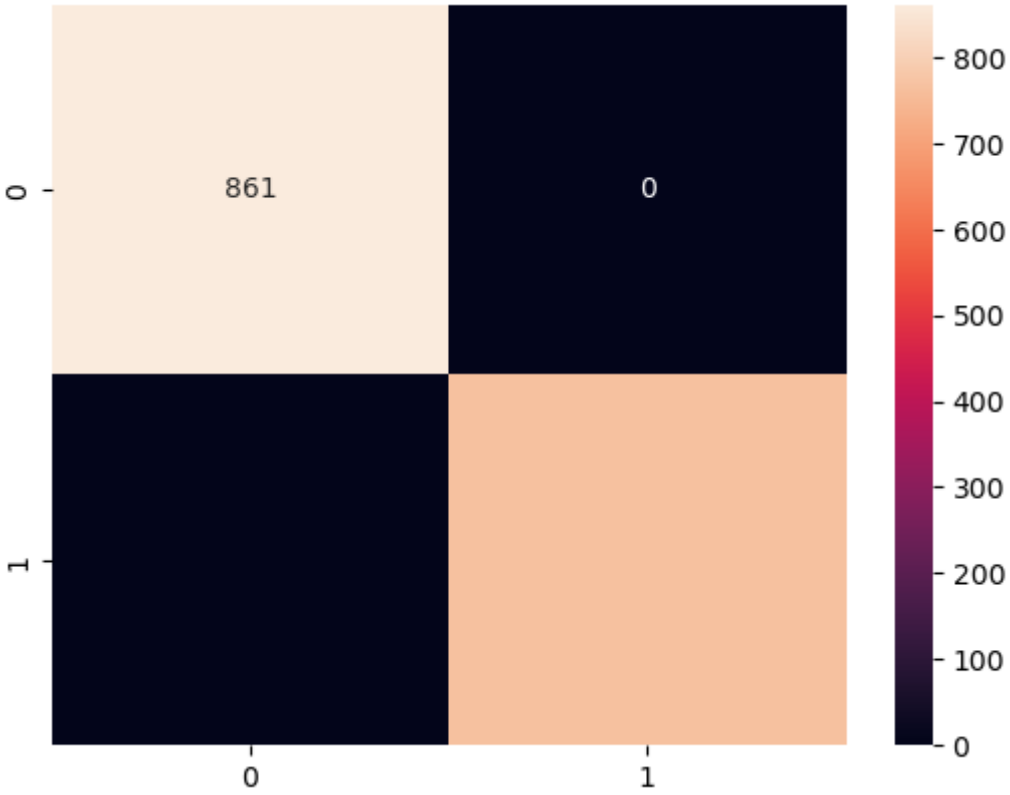
In [663…
```
decisioncm=confusion_matrix(y_test,tree_pred)
```

In [664…
```
decisioncm
```

Out[664…
```
array([[861,   0],
       [  0, 764]])
```

In [665…
```
sns.heatmap(decisioncm,annot=True,fmt='g')
```

Out[665…
```
<Axes: >
```



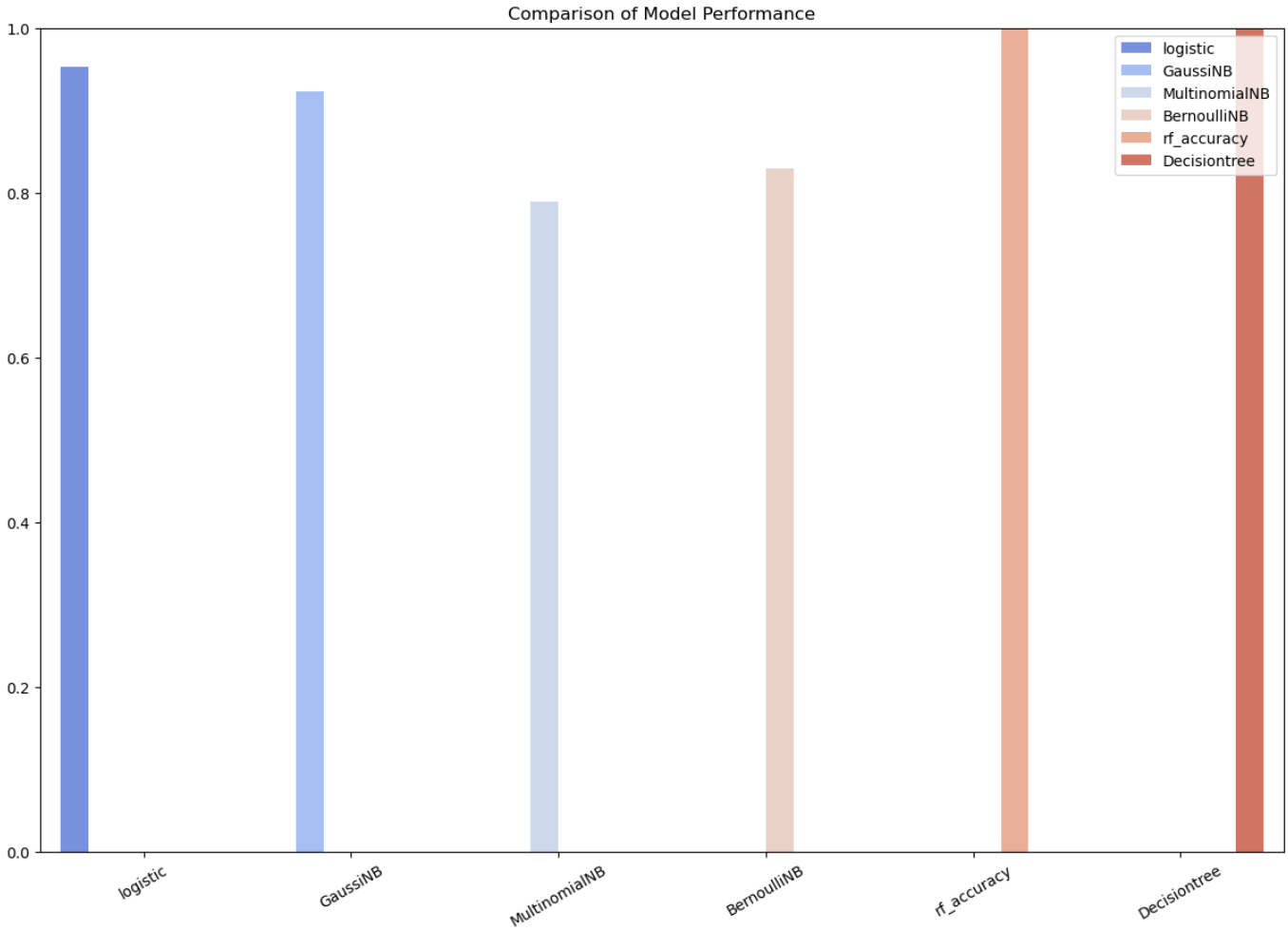
In [666…
```
models = [
    'logistic','GaussiNB','MultinomialNB','BernoulliNB','rf_accuracy','Decisiontree']
```

In [667…
```
scores = [logistic_accu,GaussiNB_accu ,MultinomialNB_accu,BernoulliNB_accu,rf_accurac
```

In [668…
```
scores
```

Out[668…
```
[0.9526153846153846,
 0.9224615384615384,
 0.7895384615384615,
 0.8295384615384616,
 1.0,
 1.0]
```

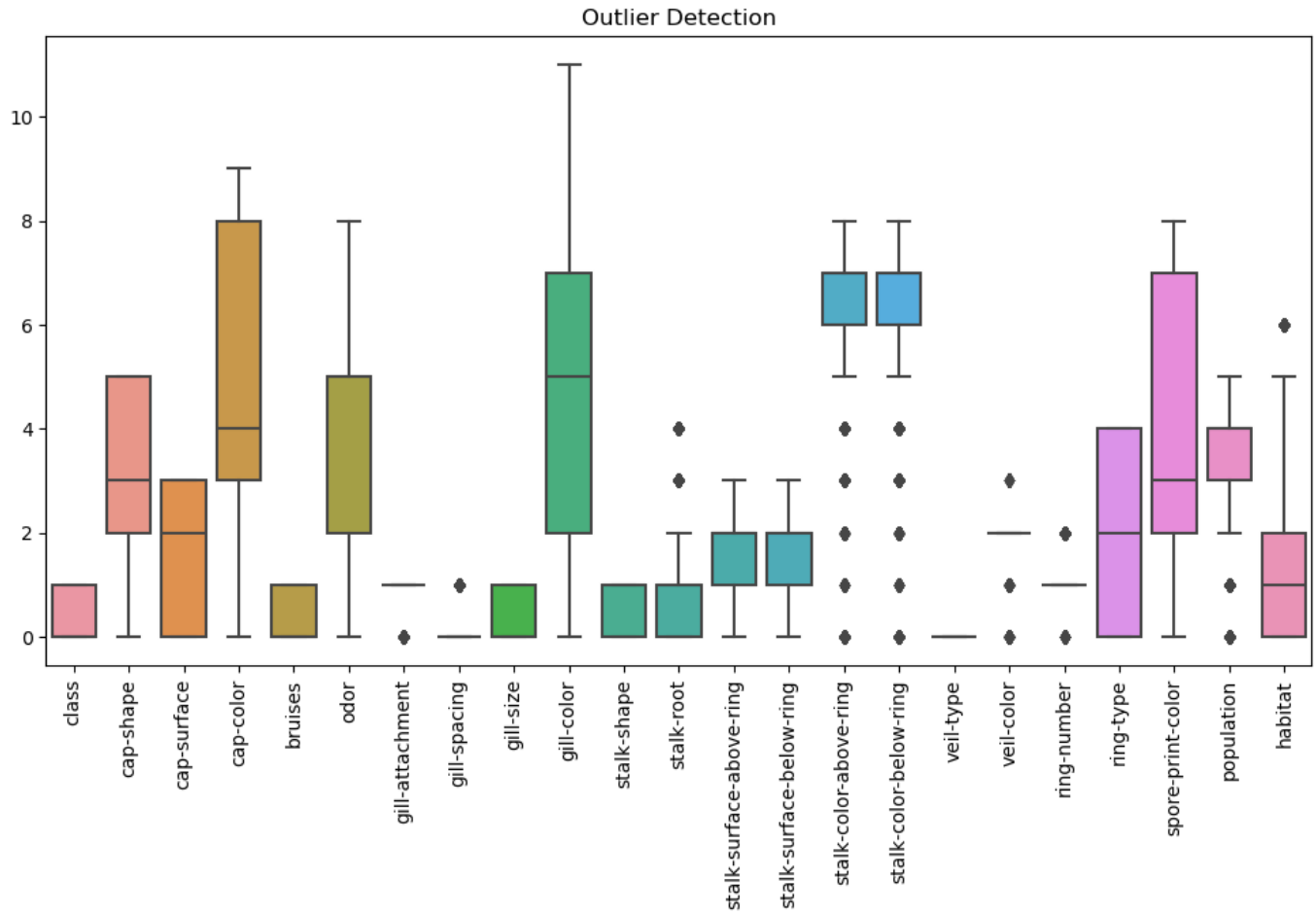
In [669…
```
plt.figure(figsize=(15, 10))
sns.barplot(x=models, y=scores, hue=models, palette='coolwarm')
plt.title("Comparison of Model Performance")
plt.xticks(rotation=30)
plt.ylim(0, 1)

plt.show()
```



Data visualization

In [704…

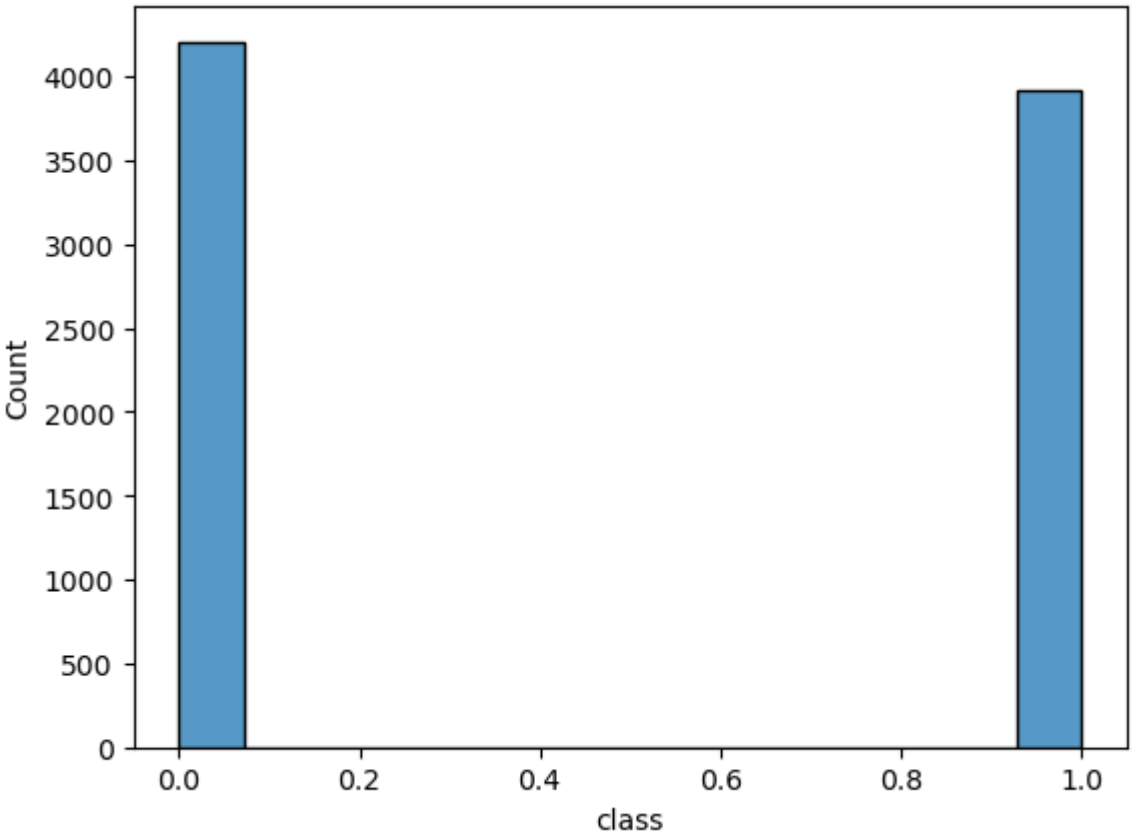
```
plt.figure(figsize=(12, 6))
sns.boxplot(data=df)
plt.xticks(rotation=90)
plt.title("Outlier Detection")
plt.show()
```



In [706…

```
sns.histplot(df["class"])
```

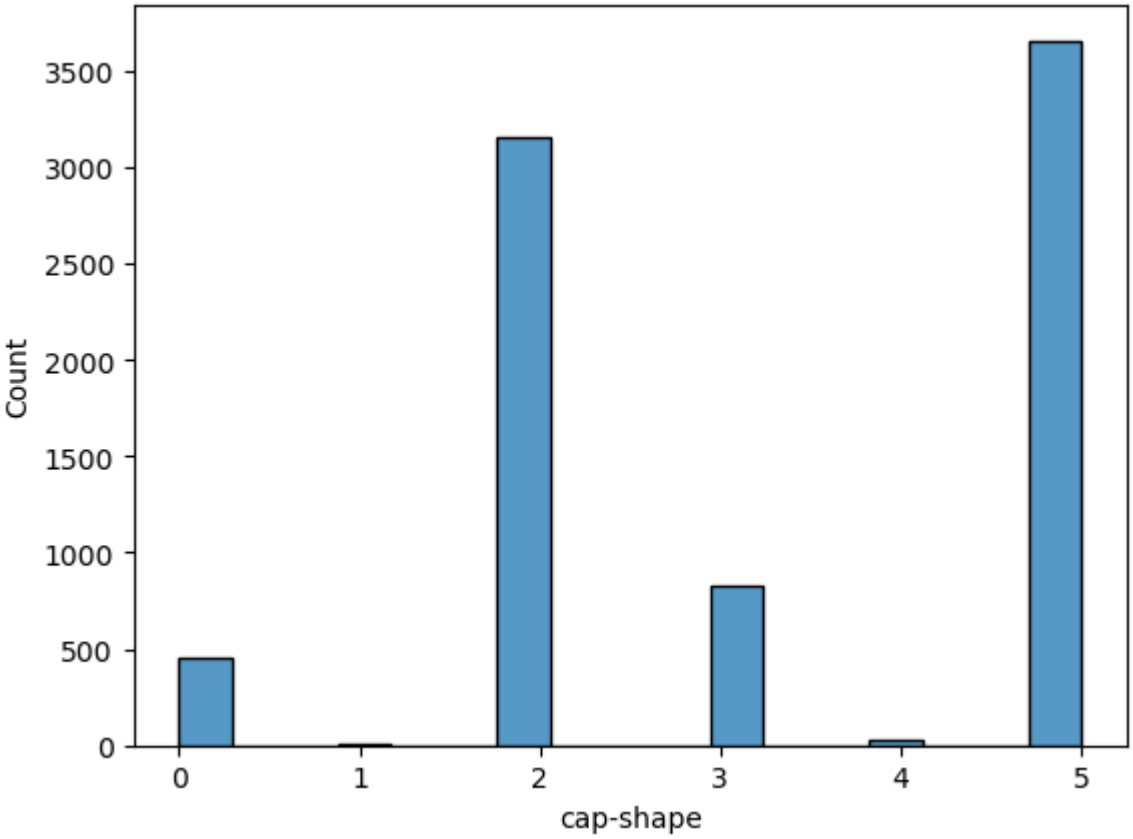
Out[706… <Axes: xlabel='class', ylabel='Count'>



In [708…

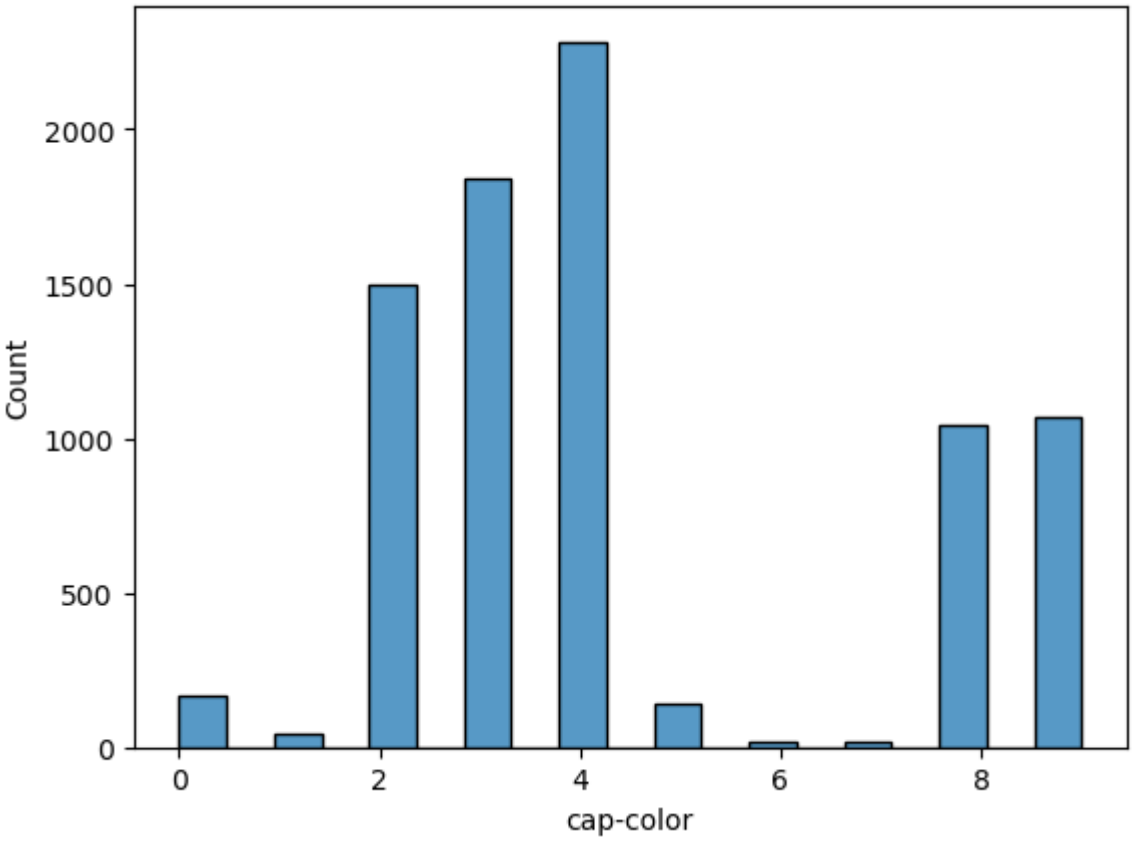
```
sns.histplot(df["cap-shape"])
```

Out[708… <Axes: xlabel='cap-shape', ylabel='Count'>



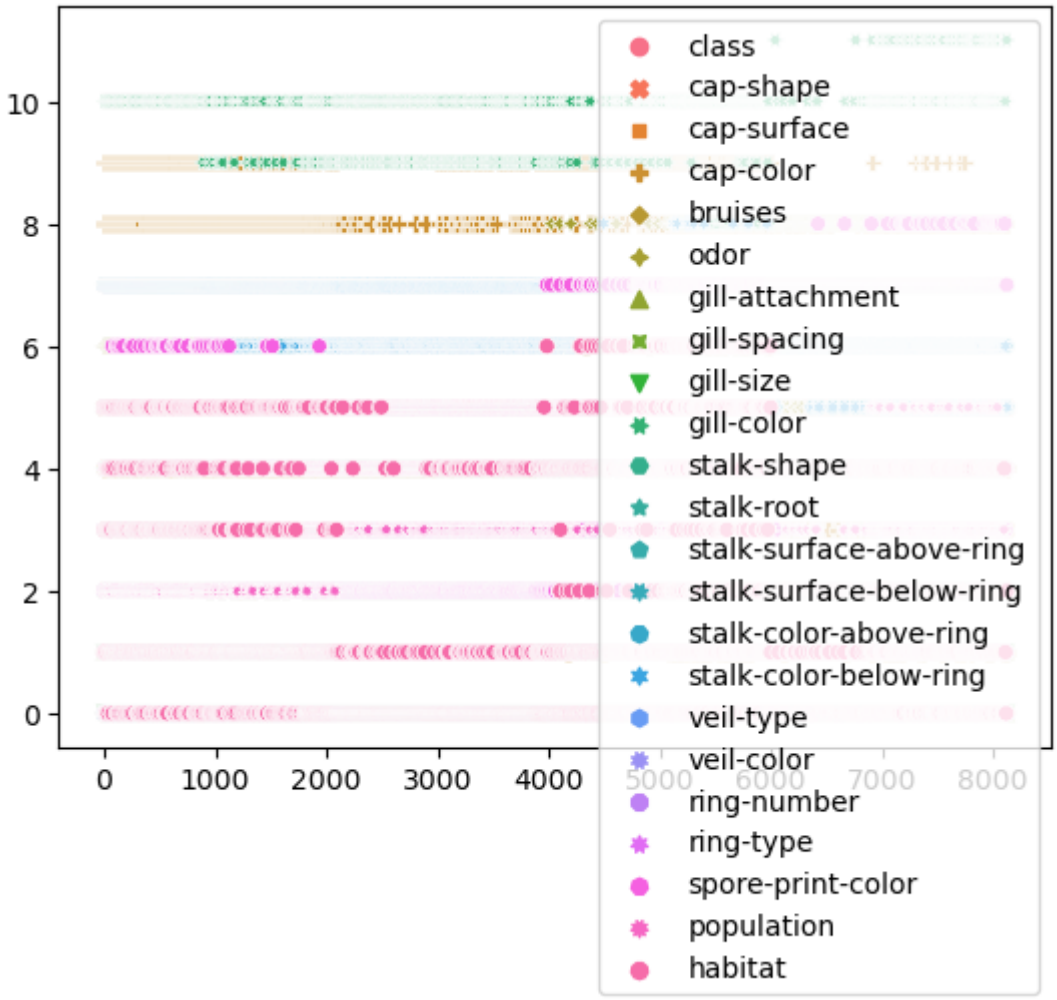
In [710…
```
sns.histplot(df["cap-color"])
```

Out[710…
```
<Axes: xlabel='cap-color', ylabel='Count'>
```



In [712…
```
sns.scatterplot(df)
plt.figure(figsize=(10, 6))
```

Out[712…
```
<Figure size 1000x600 with 0 Axes>
```



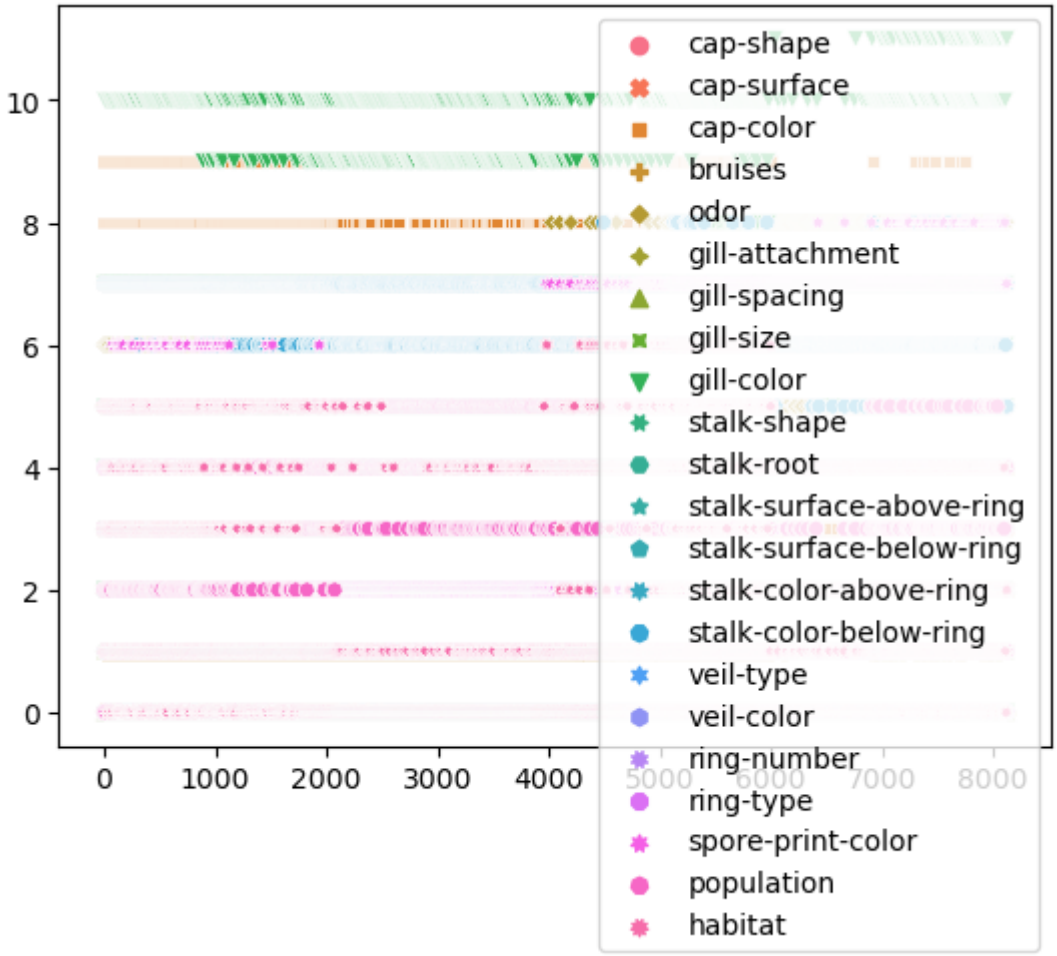

```
<Figure size 1000x600 with 0 Axes>
```

In [713…

```
sns.scatterplot(x)
```

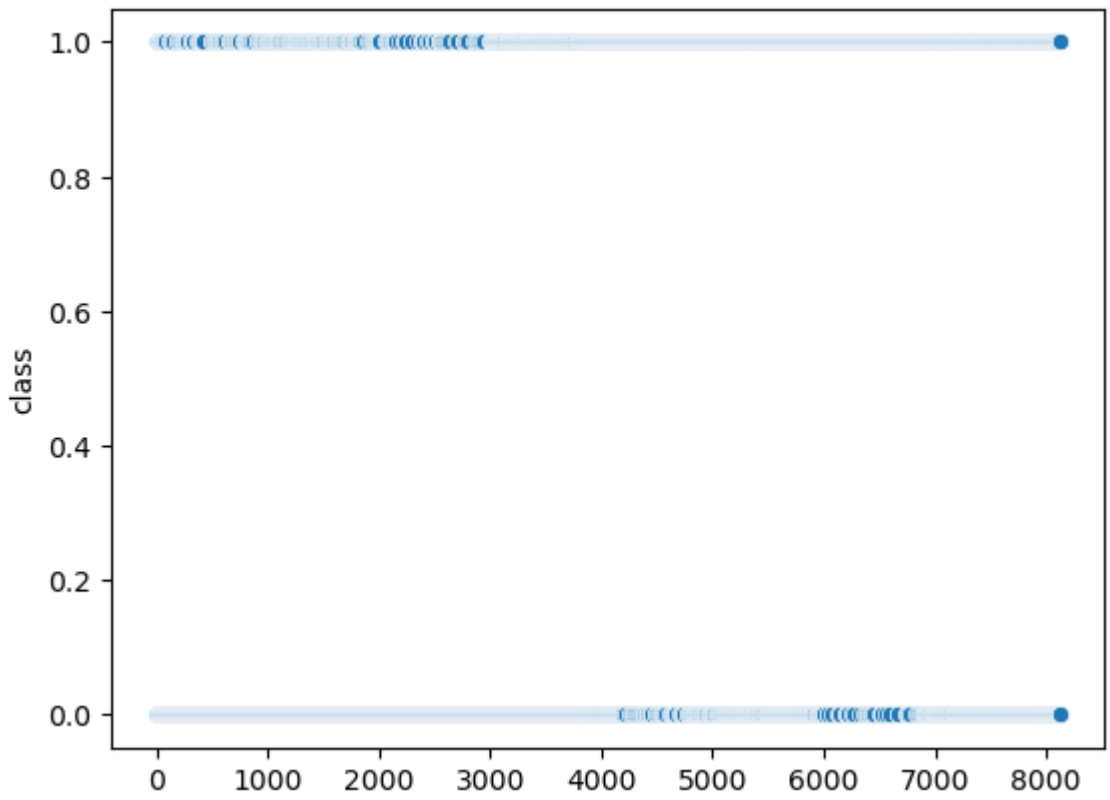
Out[713…

```
<Axes: >
```



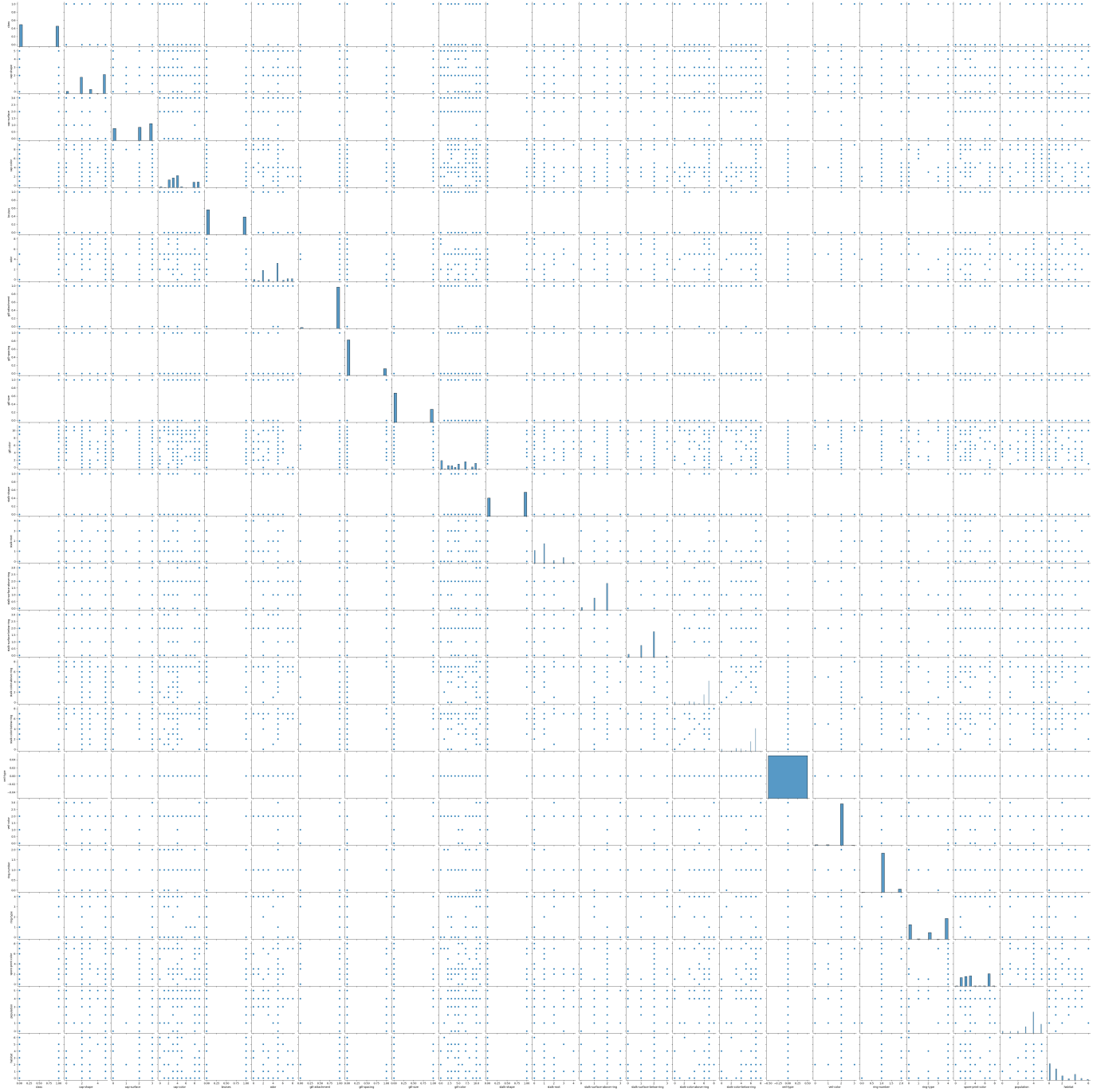
In [714…
```
sns.scatterplot(y)
```

Out[714…
```
<Axes: ylabel='class'>
```



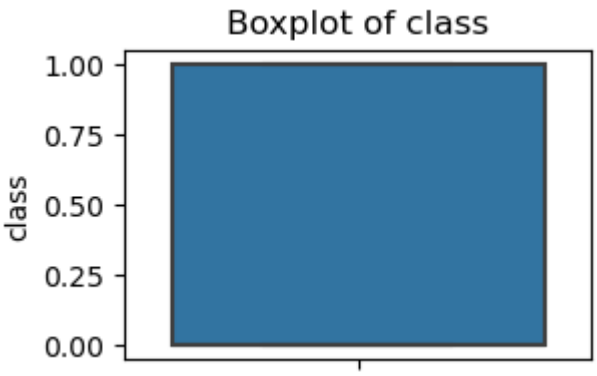
In [715…
```
sns.pairplot(df)
```

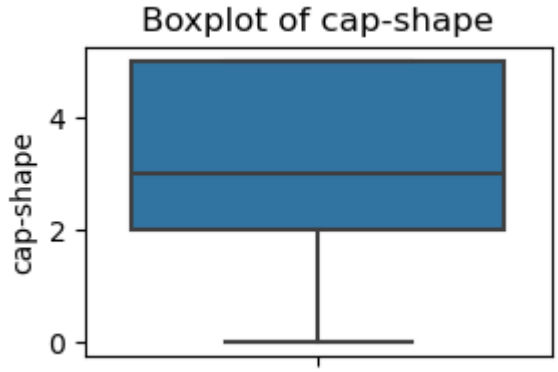
Out[715…
```
<seaborn.axisgrid.PairGrid at 0x7f5edc2eeb50>
```

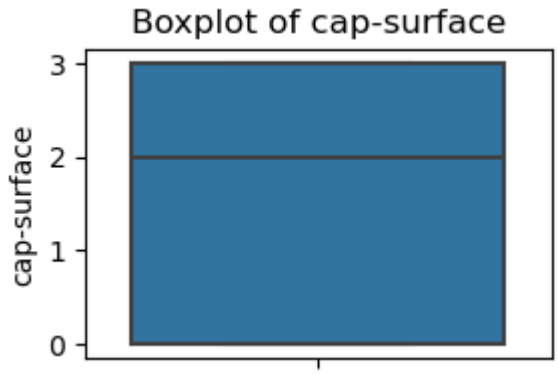


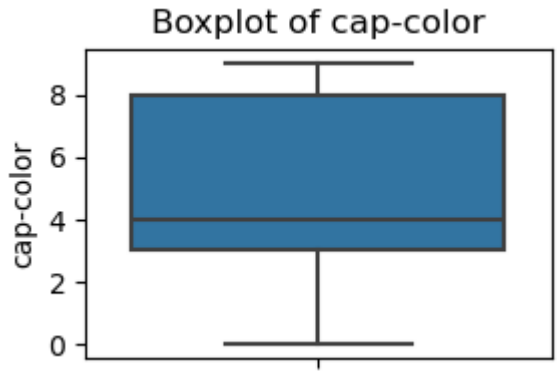
In [717…

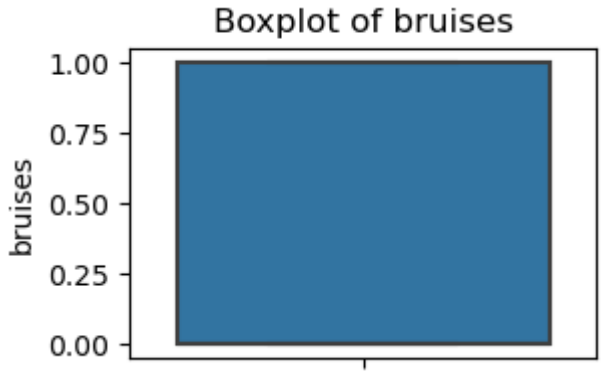
```
data=df
for feature in data:
    plt.figure(figsize=(3,2))
    sns.boxplot(y=df[feature])
    plt.title(f'Boxplot of {feature}')
    plt.show()
```

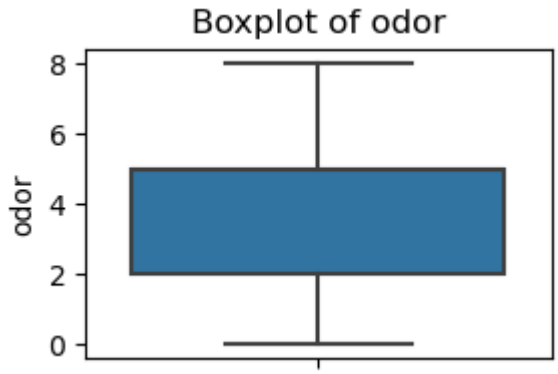


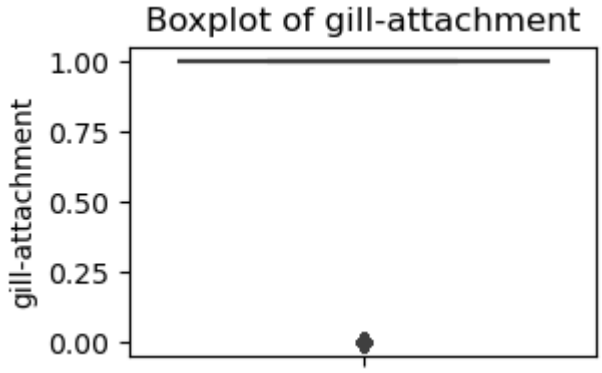

Boxplot of cap-shape
cap-shape
0
2
4


Boxplot of cap-surface
cap-surface
0
1
2
3

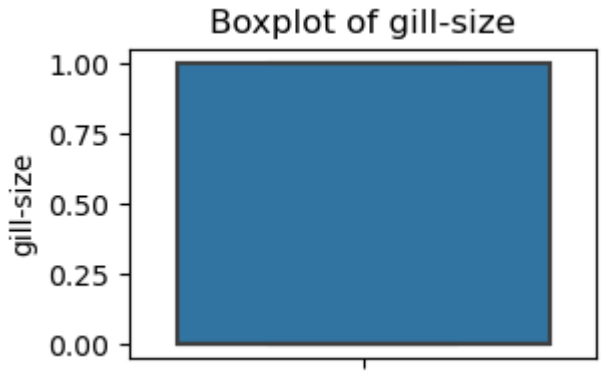

Boxplot of cap-color
cap-color
0
2
4
6
8

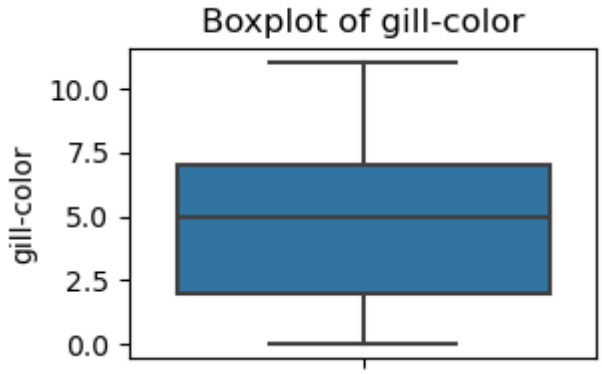

Boxplot of bruises
bruises
0.00
0.25
0.50
0.75
1.00

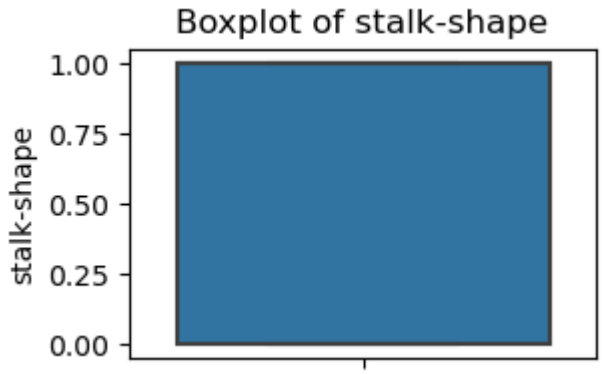

Boxplot of odor
odor
0
2
4
6
8

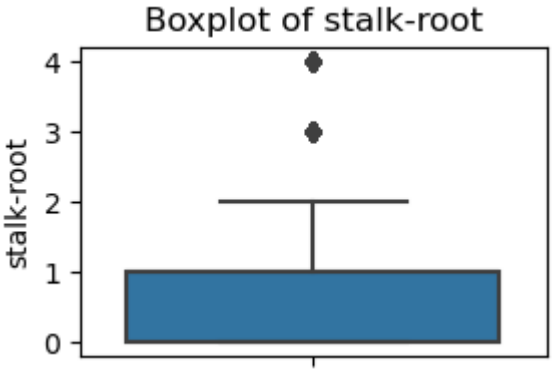

Boxplot of gill-attachment
gill-attachment
1.00
0.75
0.50
0.25
0.00

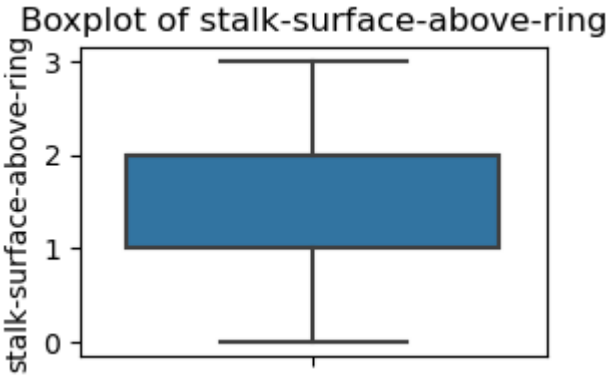
Boxplot of gill-spacing
gill-spacing
1.00
0.75
0.50
0.25
0.00

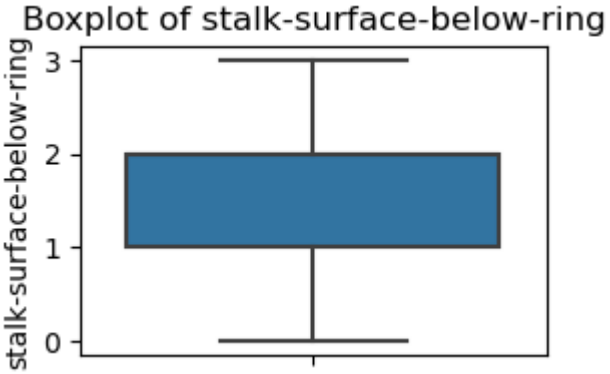

Boxplot of gill-size
gill-size
1.00
0.75
0.50
0.25
0.00

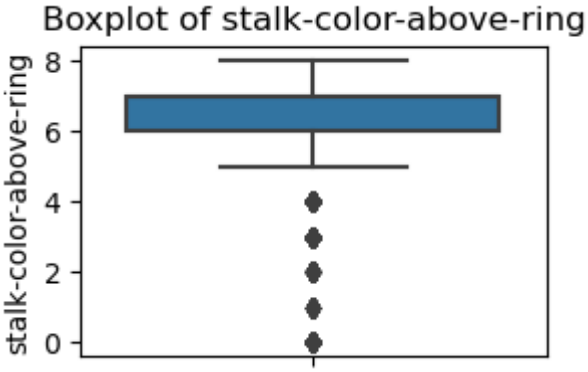

Boxplot of gill-color
gill-color
10.0
7.5
5.0
2.5
0.0

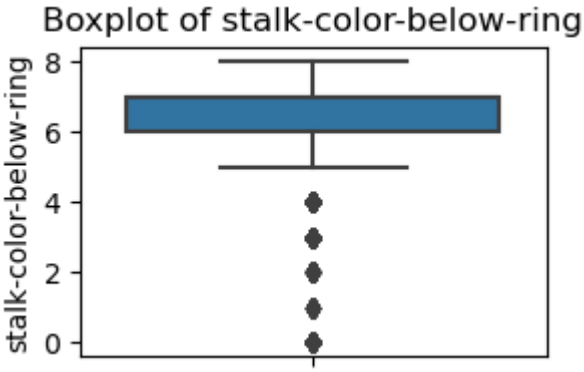

Boxplot of stalk-shape
stalk-shape
1.00
0.75
0.50
0.25
0.00

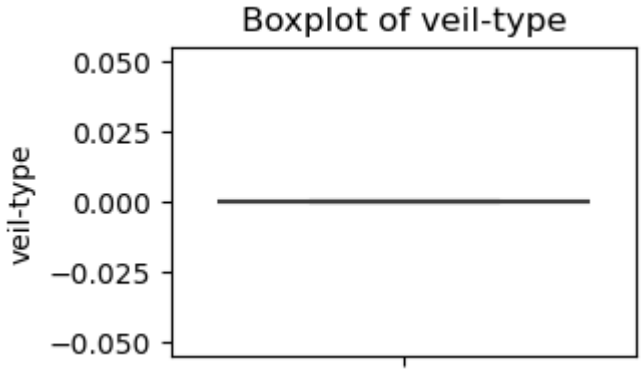

Boxplot of stalk-root
stalk-root
0
1
2
3
4

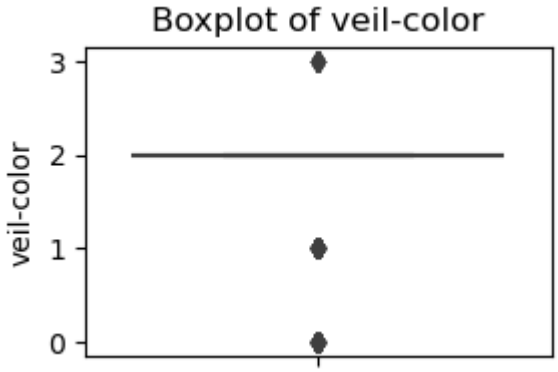

Boxplot of stalk-surface-above-ring
stalk-surface-above-ring
0
1
2
3

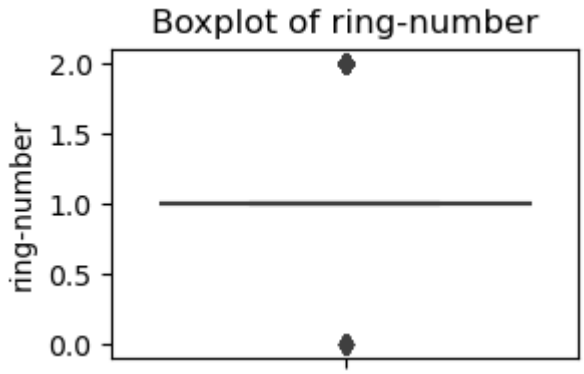

Boxplot of stalk-surface-below-ring
stalk-surface-below-ring
0
1
2
3

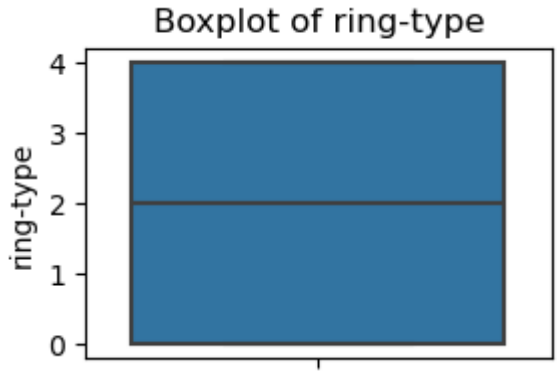

Boxplot of stalk-color-above-ring
stalk-color-above-ring
0
2
4
6
8

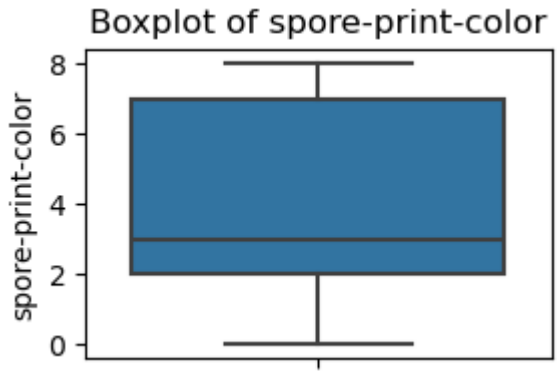

Boxplot of stalk-color-below-ring
stalk-color-below-ring
0
2
4
6
8

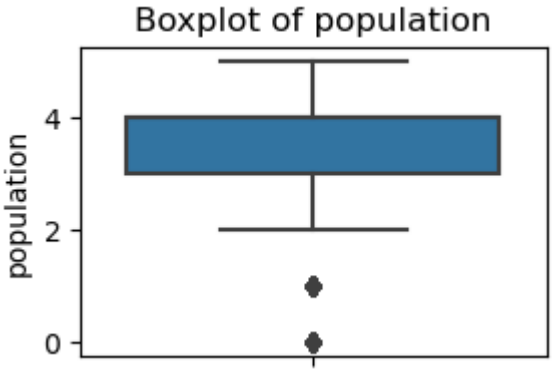

Boxplot of veil-type
veil-type
0.050
0.025
0.000
−0.025
−0.050

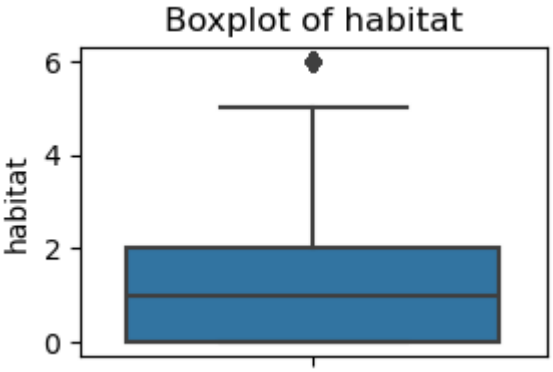

Boxplot of veil-color
veil-color
3
2
1
0


Boxplot of ring-number
ring-number
2.0
1.5
1.0
0.5
0.0


Boxplot of ring-type
ring-type
4
3
2
1
0


Boxplot of spore-print-color
spore-print-color
8
6
4
2
0


Boxplot of population
population
0
2
4


Boxplot of habitat
habitat
0
2
4
6

In [ ]:

In [ ]:

In [ ]:

In [ ]: